

Lab 3

2. Creating A Typing GUI

GridBagLayout and MigLayout in Java

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Tuesday, January 27th, 2026 | ER 3120



University of Windsor

Part I

Setting up the Environment





Setup the Environment



Setting up the JFrame and the Grid

- We set up a JFrame and create a JPanel.
- We don't use JPanel's default constructor, instead of use JPanel (LayoutManager layout) to define a Grid based layout for the components that will be added to it.
- We create a GridBagConstraints to help us describe the position of each element.
- Insets tell us the spacings/paddings between the components. In this case every element will be padded by 5 pixels from every side.

```
// We import these packages which have access to the windowing and input related classes
import java.awt.*;
import javax.swing.*;

public class Demo {

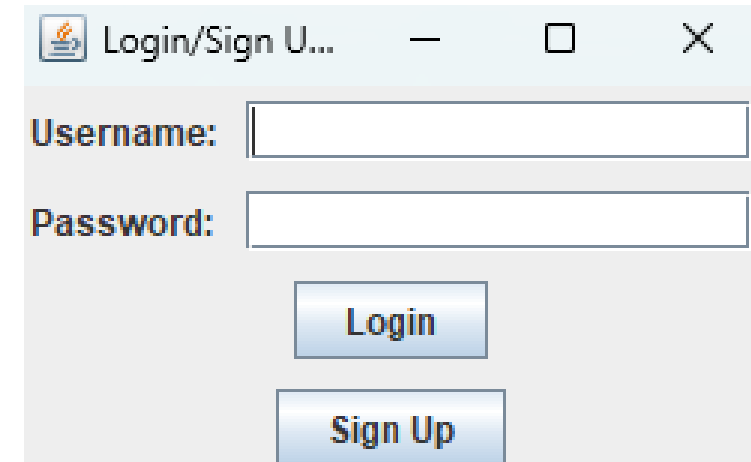
    public static void main(String[] args) {
        // Create the JFrame
        JFrame frame = new JFrame("Login/Sign Up Page");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Create the JPanel with GridBagLayout
        JPanel panel = new JPanel(new GridBagLayout());
        frame.add(panel);

        // Create GridBagConstraints for layout control
        GridBagConstraints constraints = new GridBagConstraints();
        constraints.insets = new Insets(5, 5, 5, 5); // Padding between components
    }
}
```

GridLayout vs GridBagLayout

- GridLayout makes cells of equal sizes where each component must have the same size. But what if we want some of them to have a different size, yet still be structured in a grid?
- GridBagLayout allows us to have a structured grid where a component can occupy more than 1 cell. In this case “Login” is occupying 2 cells. Similarly, “Sign Up” is also occupying 2 cells.



Adding components

- We are now trying to create the username and password labels as well as the text fields in which to type these. The gridx and gridy property tell the JPanel where to add this component in the grid layout.
- First we create the username label at (0, 0,) then we go to the next horizontal cell (1, 0) and put the username field.
- Then similarly, we create the password label in the cell below the username label (0, 1) and then add the password field next to it (1, 1).

```
// Create and add components to the panel

// Username Label
JLabel usernameLabel = new JLabel("Username:");
constraints.gridx = 0;
constraints.gridy = 0;
panel.add(usernameLabel, constraints);

// Username TextField
JTextField usernameField = new JTextField(15);
constraints.gridx = 1;
constraints.gridy = 0;
panel.add(usernameField, constraints);

// Password Label
JLabel passwordLabel = new JLabel("Password:");
constraints.gridx = 0;
constraints.gridy = 1;
panel.add(passwordLabel, constraints);

// Password PasswordField
JPasswordField passwordField = new JPasswordField(15);
constraints.gridx = 1;
constraints.gridy = 1;
panel.add(passwordField, constraints);
```

Result of previous code

- It sets a 2x2 grid since that's the amount of elements we have and plots them in their location using the coordinates we defined previously.



The image shows a Java Swing window titled "Login/Sign U...". The window has a standard title bar with a minimize button, a maximize button (disabled), and a close button. The main content area is styled with a light gray background and contains two rows of labels and text fields. The first row has a label "Username:" followed by a text field. The second row has a label "Password:" followed by a text field. The labels and text fields are arranged in a 2x2 grid layout.

Adding components continued

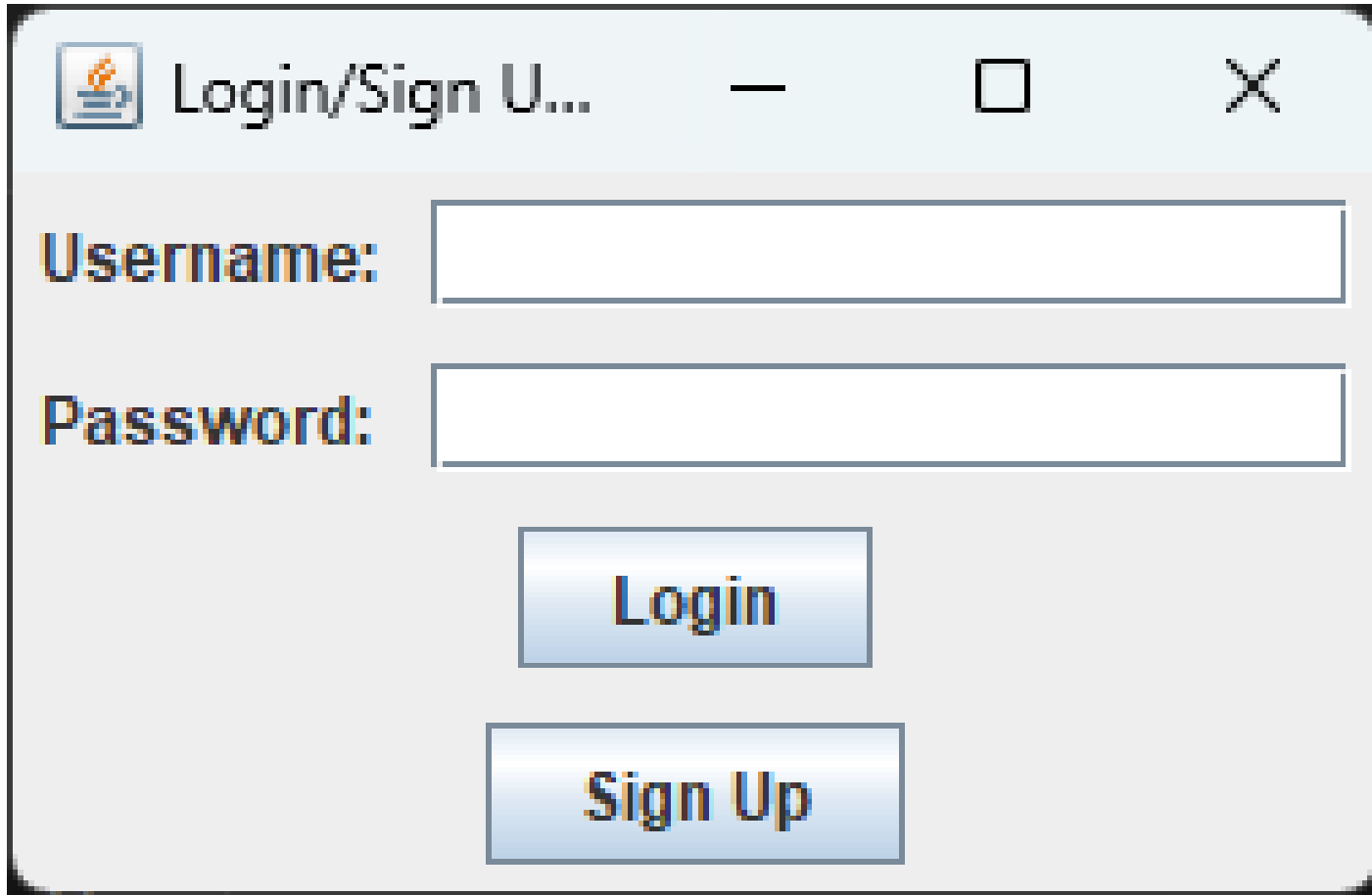
- Now we will add the login and sign up button.
- First we add the login button to (0, 2) so it appears below the password field. We set the gridwidth to 2. This means that this button will occupy 2 grid cells.
- Then we add the sign up button to (0, 3) so it appears below the login button. We also set its gridwidth to 2, so it occupies 2 grid cells.

```
// Login Button
JButton loginButton = new JButton("Login");
constraints.gridx = 0;
constraints.gridy = 2;
constraints.gridwidth = 2;
panel.add(loginButton, constraints);

// Sign Up Button
JButton signUpButton = new JButton("Sign Up");
constraints.gridx = 0;
constraints.gridy = 3;
constraints.gridwidth = 2;
panel.add(signUpButton, constraints);

frame.pack();
// Make the frame visible
frame.setVisible(true);
```


Result of previous code



The image shows a web browser window with the title 'Login/Sign U...'. Inside the window, there is a login form. The form has two input fields: 'Username:' and 'Password:'. Below the 'Password:' field, there are two buttons: 'Login' and 'Sign Up'. The window has a standard Windows-style title bar with a minimize button, a maximize button, and a close button.

Part I

The Mig Layout



The Mig Layout



● Downloading MigLayout version 11.3

○ Download the 2 jar files from:

<https://repo1.maven.org/maven2/com/miglayout/>

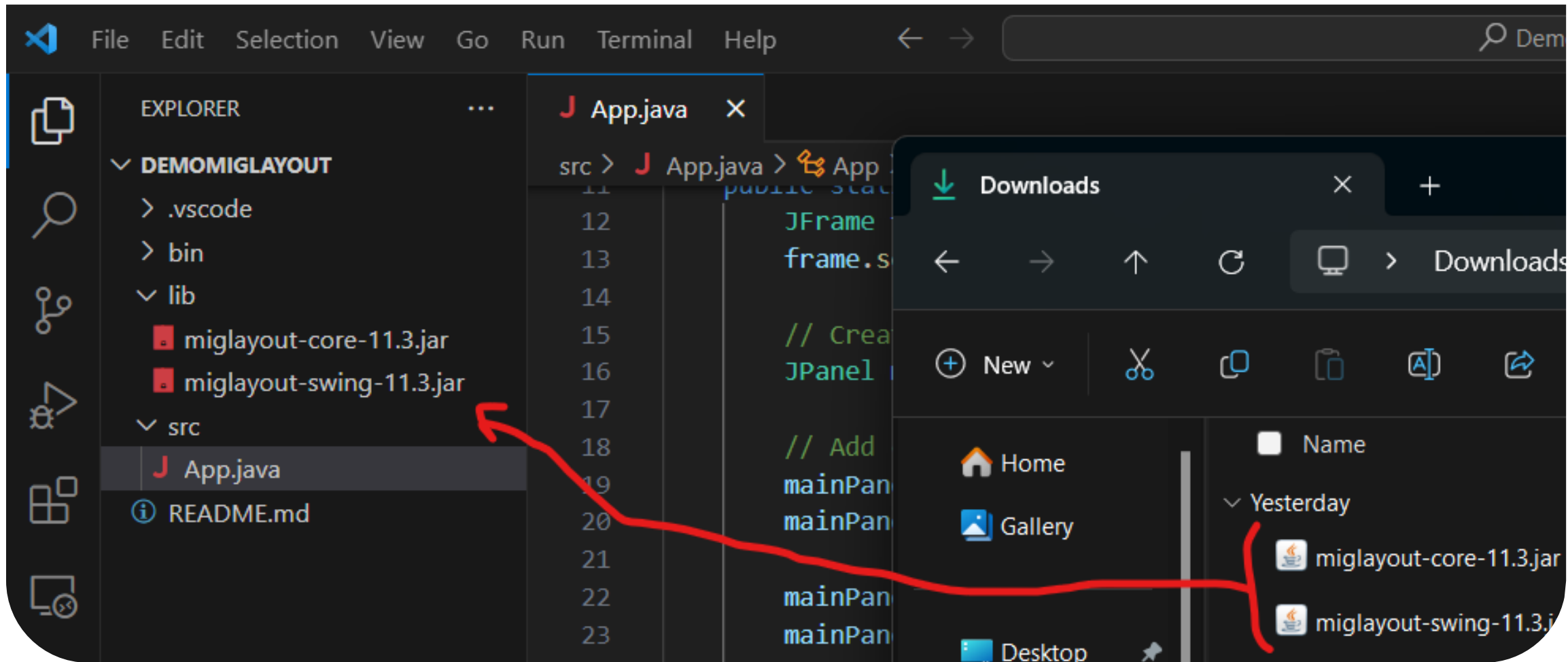


miglayout-core-11.3.jar



miglayout-swing-11.3.jar

Adding MigLayout to a project



Setting up MigLayout

```
1  import javax.swing.*;
2
3  import net.miginfocom.swing.MigLayout;
4
5  public class App {
6      public static void main(String[] args) {
7          JFrame frame = new JFrame("Sophisticated MigLayout Example");
8          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9
10         // Create the main panel with MigLayout
11         JPanel mainPanel = new JPanel(new MigLayout("alignx center, aligny center, wrap 2", "[right]15[grow,fill]"));
12
13         // Set the main panel as the content pane of the frame
14         frame.setContentPane(mainPanel);
15
16         // Pack and display the frame
17         frame.pack();
18         frame.setLocationRelativeTo(null);
19         frame.setVisible(true);
20     }
21 }
```

Adding some components

- This will create a row of 12 buttons. We haven't specified any organization parameters, and hence this is the way MigLayout will display the buttons by default.

```
import javax.swing.*;
import net.miginfocom.swing.MigLayout;

public class App {
    public static void main(String[] args) {
        JFrame frame = new JFrame("MigLayout Example");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

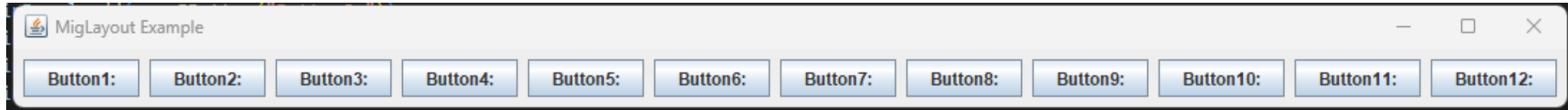
        // Create the main panel with MigLayout
        JPanel mainPanel = new JPanel(new MigLayout());

        // Add components to the panel
        mainPanel.add(new JButton("Button1:"));
        mainPanel.add(new JButton("Button2:"));
        mainPanel.add(new JButton("Button3:"));
        mainPanel.add(new JButton("Button4:"));
        mainPanel.add(new JButton("Button5:"));
        mainPanel.add(new JButton("Button6:"));
        mainPanel.add(new JButton("Button7:"));
        mainPanel.add(new JButton("Button8:"));
        mainPanel.add(new JButton("Button9:"));
        mainPanel.add(new JButton("Button10:"));
        mainPanel.add(new JButton("Button11:"));
        mainPanel.add(new JButton("Button12:"));

        // Add the panel to the frame
        frame.add(mainPanel);

        // Pack and display the frame
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

Output

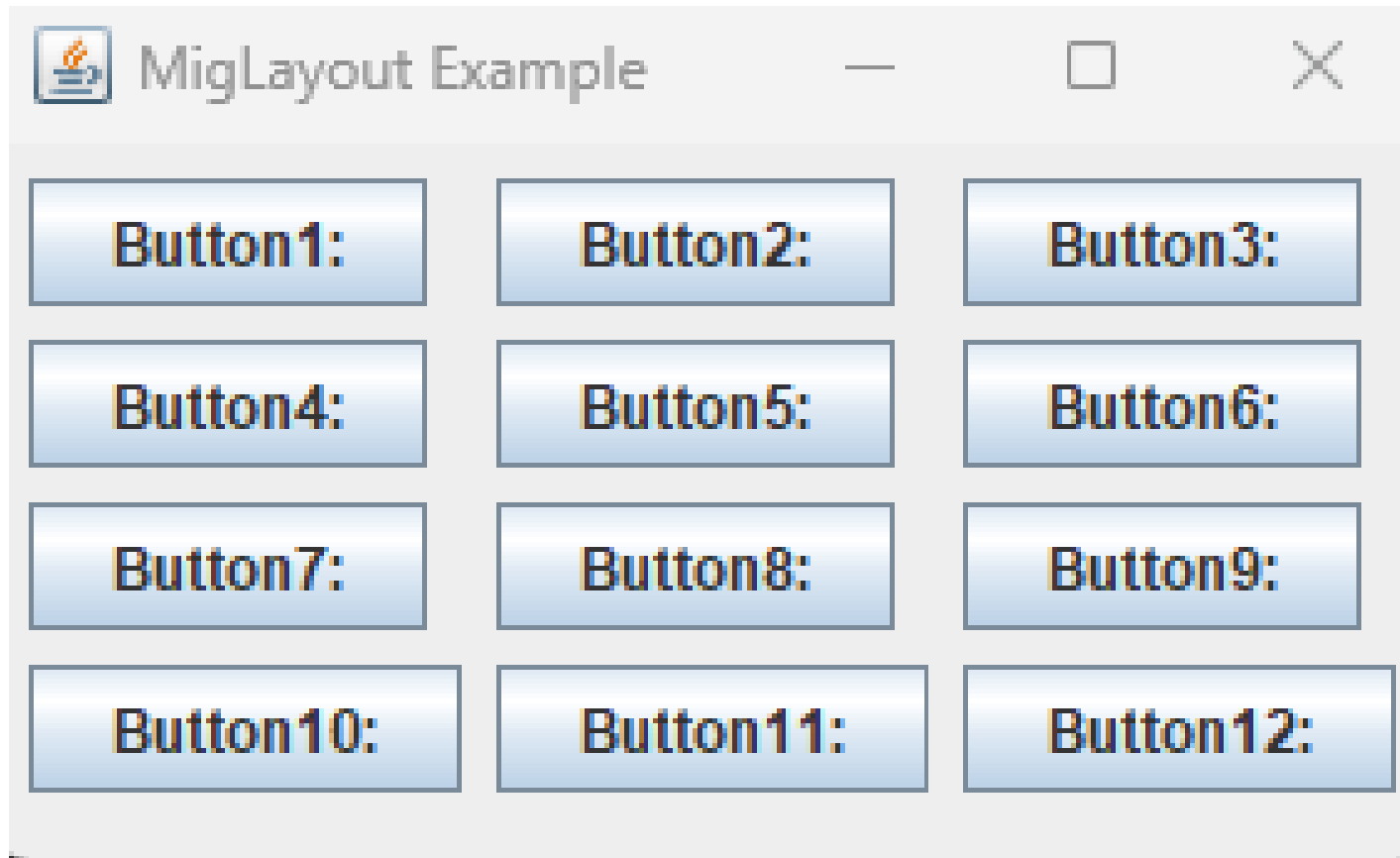


Adding rows (wrapping)

- Adding the “wrap” property allows the following components to be added to a new row.
- So, in this case after the 3rd button, the 4th button will be added to a new row. Same with the 6th button and 9th and 12th.
- So, in this case we will have 3 columns and 4 rows.

```
// Add components to the panel
mainPanel.add(new JButton("Button1:"));
mainPanel.add(new JButton("Button2:"));
mainPanel.add(new JButton("Button3:"), "wrap");
mainPanel.add(new JButton("Button4:"));
mainPanel.add(new JButton("Button5:"));
mainPanel.add(new JButton("Button6:"), "wrap");
mainPanel.add(new JButton("Button7:"));
mainPanel.add(new JButton("Button8:"));
mainPanel.add(new JButton("Button9:"), "wrap");
mainPanel.add(new JButton("Button10:"));
mainPanel.add(new JButton("Button11:"));
mainPanel.add(new JButton("Button12:"), "wrap");
```

Output

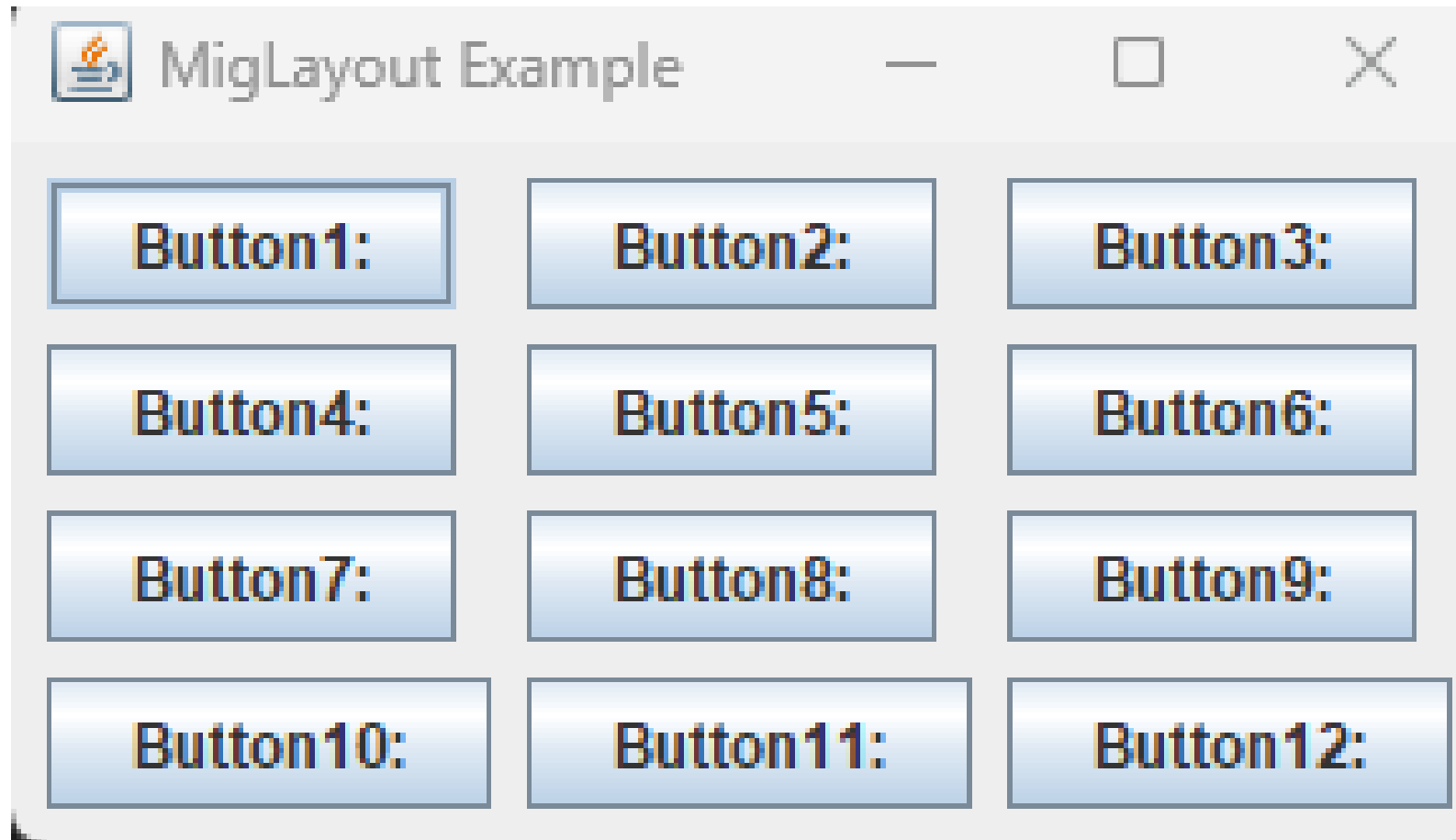


Another way to manage columns and rows.

```
// Create the main panel with MigLayout
JPanel mainPanel = new JPanel(new MigLayout("wrap", "[[][]]", "[[][]][[]]"));

// Add components to the panel
mainPanel.add(new JButton("Button1:"));
mainPanel.add(new JButton("Button2:"));
mainPanel.add(new JButton("Button3:"));
mainPanel.add(new JButton("Button4:"));
mainPanel.add(new JButton("Button5:"));
mainPanel.add(new JButton("Button6:"));
mainPanel.add(new JButton("Button7:"));
mainPanel.add(new JButton("Button8:"));
mainPanel.add(new JButton("Button9:"));
mainPanel.add(new JButton("Button10:"));
mainPanel.add(new JButton("Button11:"));
mainPanel.add(new JButton("Button12:"));
```

Output:



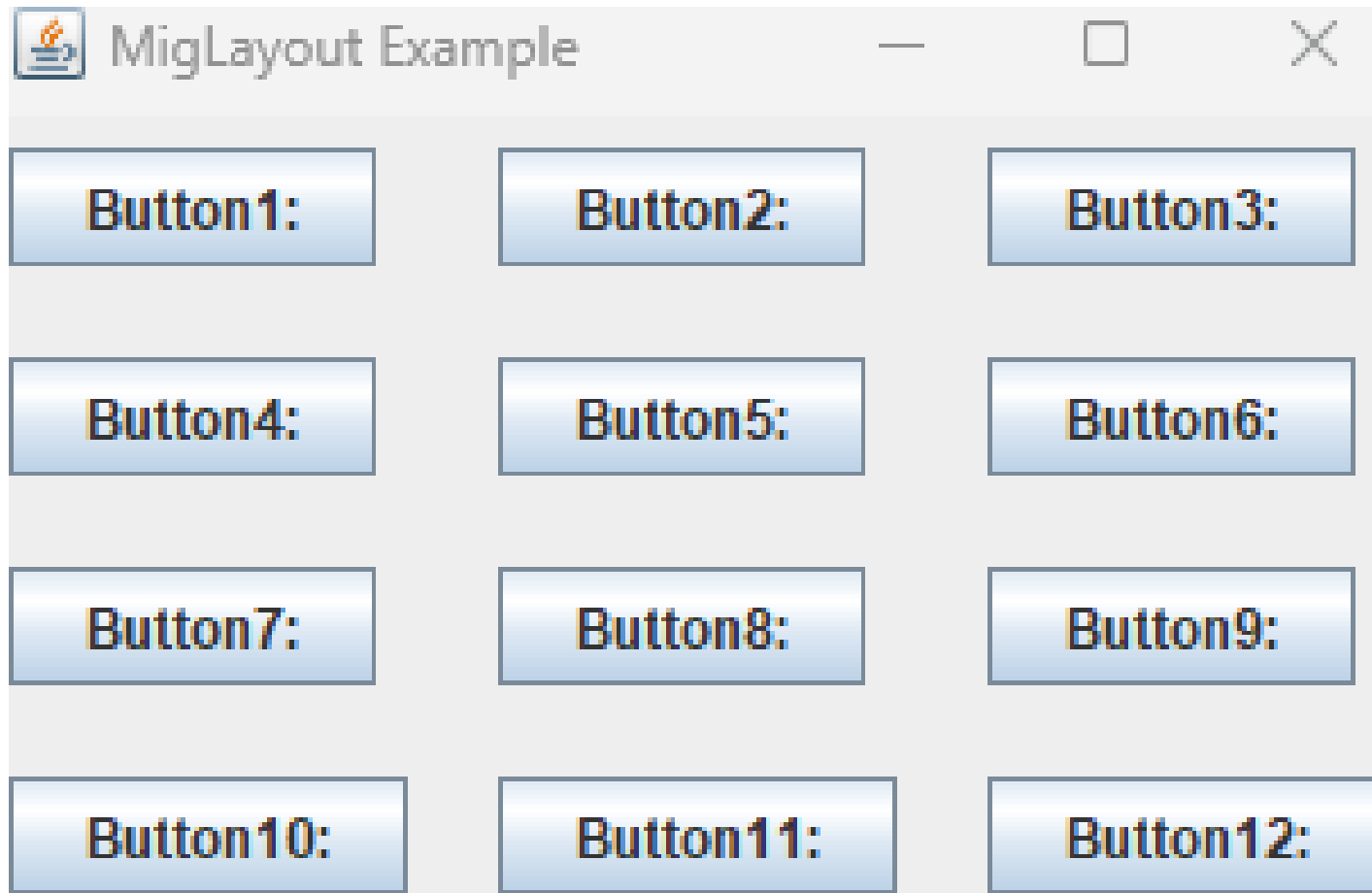
Specifying spacing between cells.

- MigLayout allows us to easily define the spacing between each column and each row.
- You can specify the spacing between each cell as follows. (spacing is given in number of pixels, if they are not specified, by default it is 5 pixels).
- This code will add 20 pixels of spacing between each cell.

```
// Create the main panel with MigLayout
JPanel mainPanel = new JPanel(new MigLayout("wrap", "[20][20[]", "[20][20[]20[]"));

// Add components to the panel
mainPanel.add(new JButton("Button1:"));
mainPanel.add(new JButton("Button2:"));
mainPanel.add(new JButton("Button3:"));
mainPanel.add(new JButton("Button4:"));
mainPanel.add(new JButton("Button5:"));
mainPanel.add(new JButton("Button6:"));
mainPanel.add(new JButton("Button7:"));
mainPanel.add(new JButton("Button8:"));
mainPanel.add(new JButton("Button9:"));
mainPanel.add(new JButton("Button10:"));
mainPanel.add(new JButton("Button11:"));
mainPanel.add(new JButton("Button12:"));
```

Output





Useful resources:

- <https://www.javatpoint.com/java-gridbaglayout>
- <https://www.miglayout.com/>
- <https://www.miglayout.com/mavensite/docs/QuickStart.pdf>
- <https://www.migcalendar.com/miglayout/mavensite/docs/cheatsheet.pdf>
- <https://www.migcalendar.com/miglayout/mavensite/docs/whitepaper.html>
- <http://www.migcalendar.com/miglayout/mavensite/apidocs/index.html>



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor